Business Problem Justification

Problem: Brokerage firms lose money when macroeconomic news is released due to sharp price fluctuations and their inability to execute client orders in a timely manner.

Solution: Our project analyzes historical data from APIs (prices) and web scraping (news) to identify the types of events that have the greatest impact. This will enable us to automate our risk management system: the program will automatically "detect" important news events on the calendar and widen spreads or lower limits in advance, thereby protecting the company's capital.

This enables:

- risk management around high-impact news releases
- data-driven trading signals based on Actual vs Forecast deviations
- ranking of event types by their real effect on a specific asset

Data Sources:

- Forex Factory - https://www.forexfactory.com/ — scraped: structured historical event data (Impact, Actual, Forecast, Previous)
- Alpha Vantage https://www.alphavantage.co/ — API: official financial data provider for daily price history

Justification:

When big news comes out like a report on jobs or a decision by a bank the market can change very quickly. The people who make the markets called market makers stop trading for a moment. The difference between the prices they are willing to buy and sell at gets much bigger. This means that the brokerage firms are stuck with the obligation to buy or sell at the prices, which can be very unpleasant for them. They also have trades that're already open and these can be affected very quickly by the news before anyone can even react. All of this happens fast and it can cause big losses for the firms.

Some types of news like reports on the economy or decisions by central banks are more likely to cause big changes than others. We can also see that different currencies are affected differently by types of news. This means that the firms need to have a way to react to each type of news differently than just having a blanket policy.

We can use data from the past to predict which news releases are likely to cause changes in the market. We do this by looking at how the market has reacted to news in the past. Then we use this information to adjust the prices and limits before the news is released. This way the firms can avoid the losses that happen when they are not prepared.

This solution can help the firms in three ways. First, it can be helpful to avoid losing money when the market changes quickly. Second, it can help them use their capital efficiently by only taking defensive measures when they are really needed. Third, it can help firms be more resilient by removing the need for individual traders to make decisions in reaction to news. This makes the whole process more consistent and reliable.

Brokerage firms that deal with exchange and cryptocurrency markets need to change the way they do things. They need to use data and automation to react to news releases. This

is the way they can avoid big losses and stay competitive. The old way of doing things is not efficient enough anymore.

The use of automation to react to news releases is the key to avoiding losses and being successful, in currency markets. Firms need to use technology to stay ahead of the game.

Data Sources for EDA: Data is pulled from a PostgreSQL database (`hse_parsing`) with four tables:

`events` - Core macroeconomic event records with scheduled release values

`event_specs` - Metadata and descriptive attributes for each event type

`event_history` - Historical time-series of actual/forecast/previous values per event

`currency_ohlc` - Daily OHLC currency price data (USD-quoted pairs)

Raw Table Schemas and Features

Table: `events`

The primary events table. Each row is a single scheduled macroeconomic event release.

Column	Type	Description
`id`	int	Primary key for the event record
`datetime`	datetime	Exact scheduled date and time of the release (UTC)
`currency`	str	ISO currency code of the country the event belongs to (e.g., `USD`, `EUR`, `GBP`)
`impact`	str	Textual impact label from the source (`High Impact Expected`, `Medium Impact Expected`, `Low Impact Expected`, `Non-Economic`)
`title`	str	Name/title of the economic indicator (e.g., `Non-Farm Payrolls`, `CPI m/m`)

`actual`	**str (raw)**	**Released actual value as a raw string (e.g., `"2.3%"`, `"1.2M"`, `"-5K"`)**
`forecast`	**str (raw)**	**Analyst consensus forecast value as a raw string**
`previous`	**str (raw)**	**Previously reported value as a raw string**
`currency_full`	**str**	**Full currency name (dropped in final dataset)**
`speaker`	**str**	**Speaker name for speech/press conference events (dropped in final dataset)**
`acro_expand`	**str**	**Acronym expansion for the indicator (dropped in final dataset)**
`also_called`	**str**	**Alternative names for the indicator (dropped in final dataset)**

Derived columns added during preprocessing:

Column	Type	Description
`event_date`	date	Calendar date extracted from `datetime`
`impact_level`	int	Numeric encoding of impact: `3` = High, `2` = Medium, `1` = Low, `0` = Non-Economic, `NaN` = unknown

`actual_num`	float	Parsed numeric value of `actual` (with unit multipliers resolved, e.g. `1.2M` - `1200000.0`)
`forecast_num`	float	Parsed numeric value of `forecast`
`previous_num`	float	Parsed numeric value of `previous`
`has_actual`	bool	`True` if `actual_num` is not NaN
`has_forecast`	bool	`True` if `forecast_num` is not NaN
`has_previous`	bool	`True` if `previous_num` is not NaN
`is_quantitative`	bool	`True` if at least one of actual/forecast/previous is numeric
`is_speech_like`	bool	`True` if title contains speech keywords OR all three numeric fields are NaN
`event_type`	str	Keyword-based classification of the event (see Event Type Classification below)
`signal`	float	Directional signal: `1` = positive for currency, `-1` = negative, `0` = neutral, `NaN` = not computable
`actual_final`	float	`actual_num` filled from `event_history` where missing
`forecast_final`	float	`forecast_num` filled from `event_history` where missing
`previous_final`	float	`previous_num` filled from `event_history` where missing

Table: `event_specs`

Long-format metadata table describing each unique event type. One event can have many spec rows (key-value pairs).

Column	Type	Description
`event_id`	int	Foreign key referencing `events.id`
`label`	str	Spec attribute name (e.g., `"Usual Effect"`, `"Measures"`, `"Frequency"`, `"Source"`)
`value`	str	Attribute value (free text)

Normalization: Labels are lowercased and mapped to a controlled vocabulary. The table is pivoted from long to wide format using `event_id` as the index.

Pivoted columns (specs_wide_df):

Column	Description
`usual_effect`	Text description of the typical market impact (e.g., `"Greater than forecast is good for currency"`)
`measures`	What the indicator measures (e.g., `"Change in the number of employed people"`)
`frequency`	Release frequency (e.g., `Monthly`, `Quarterly`)
`source`	Issuing institution (e.g., `Bureau of Labor Statistics`)
`ff_notes`	ForexFactory editorial notes about the indicator
`next_release`	Scheduled date of the next release
`why_traders_care`	Editorial explanation of market relevance (dropped in final dataset)
`derived_via`	How the figure is calculated (dropped in final dataset)

`description`	Long description of the indicator (dropped in final dataset)
`effect_rule`	Parsed from `usual_effect`: one of `gt_good`, `lt_good`, `gt_bad`, `lt_bad`, or `NaN`

Table: `event_history`

Time-series of historical releases for each event type. Used to backfill missing actual/forecast/previous values in the main events table.

Column	Type	Description
`event_id`	int	Foreign key referencing `events.id`
`date`	date	Date of the historical release
`actual`	str (raw)	Historical actual value as raw string
`forecast`	str (raw)	Historical forecast value as raw string
`previous`	str (raw)	Historical previous value as raw string
`actual_num`	float	Parsed numeric actual (same `parse_value` logic as events)
`forecast_num`	float	Parsed numeric forecast
`previous_num`	float	Parsed numeric previous

Usage: Only the most recent historical record per `event_id` is used (`history_last_df`), and only to fill gaps in the events table's `actual_num`, `forecast_num`, `previous_num` columns.

Table: `currency_ohlc`

Daily OHLC candlestick data for Forex pairs (all quoted vs. USD).

Column	Type	Description
`bar_date`	date	Trading day
`from_currency`	str	Base currency ISO code (e.g., `EUR`, `GBP`, `JPY`)
`open`	float	Opening price of the currency pair on `bar_date`
`high`	float	Intraday high price
`low`	float	Intraday low price
`close`	float	Closing price

Currencies covered: AUD, GBP, CAD, NZD, EUR, JPY, CHF (all vs. USD).

Note on USD: Since no `USD/USD` pair exists, USD events are matched to EUR/USD data with the return sign inverted (i.e., a stronger USD means a falling EUR/USD, so return = −(close − open) / open).

Final Analytical Dataset (`df_final`)

The final dataset is constructed by:

1. Aggregating `events` to daily level per currency (`df_daily`)
2. Merging with `currency_ohlc` to attach price returns and volatility
3. Dropping rows with missing return or volatility (~8%, corresponding to non-trading days such as public holidays)

Features in `df_final`

Column	Type	Description

`event_date`	date	Calendar date
`currency`	str	ISO currency code
`event_count`	int	Total number of economic events scheduled for this currency on this date
`impact_sum`	float	Sum of `impact_level` values across all events that day
`impact_mean`	float	Mean `impact_level` across all events that day
`inflation`	int	Count of inflation-type events (CPI, PPI, price indices)
`labor`	int	Count of labor-type events (employment, unemployment, payrolls, wages)
`rates`	int	Count of monetary policy events (rate decisions, central bank statements)
`macro`	int	Count of macro output events (GDP, growth)
`consumption`	int	Count of consumption events (retail sales, consumer spending, income)
`business`	int	Count of business activity events (PMI, manufacturing, industrial production)
`sentiment`	int	Count of sentiment/survey events (confidence indices, surveys)
`other`	int	Count of events not matching the above categories
`signal_mean`	float	Mean directional signal for the day (0 = neutral or no signal available)
`open`	float	Opening FX rate for the currency on this date

`high`	float	Intraday high FX rate
`low`	float	Intraday low FX rate
`close`	float	Closing FX rate
`from_currency`	str	Base currency of the matched OHLC pair (same as `currency` for non-USD; `EUR` for USD rows)
`return`	float	Daily return: `(close - open) / open`; negated for USD
`volatility`	float	Intraday range as a fraction of open: `(high - low) / open`

Target Variables

The dataset supports two predictive tasks:

- **Volatility prediction (regression):** Predict `volatility` from news features. This is the primary target, motivated by the observation that extreme volatility spikes are universal across currencies and appear correlated with high-impact news days.
- **Return direction prediction (classification):** Predict the sign of `return` using `signal_mean` and event-type counts.

Event Type Classification

Events are classified by keyword matching on a combined string of `title` and `measures`:

Class	Keywords matched	Examples
`inflation`	cpi, inflation, price, ppi	CPI m/m, Core PCE Price Index
`labor`	employment, unemployment, job, payroll, wage	Non-Farm Payrolls, Unemployment Rate

`rates`	rate, fed, ecb, fomc, central bank, interest	Fed Funds Rate, ECB Rate Decision
`macro`	gdp, growth, output	GDP q/q, Preliminary GDP
`consumption`	retail, consumer, spending, income	Retail Sales m/m, Personal Spending
`business`	pmi, manufacturing, industrial, production, business	Manufacturing PMI, Industrial Production
`sentiment`	confidence, sentiment, survey	Consumer Confidence, ZEW Economic Sentiment
`other`	(no match)	Press conferences, minor statistical releases

Missing Values

Raw Events Table

Column	Missing values origin	Handling
`actual`	Event not yet released at parse time, or speech events with no numeric value	Treated as `NaN`; backfilled from `event_history` where available
`forecast`	Not all events have analyst forecasts	Treated as `NaN`; backfilled from `event_history`
`previous`	Rare; typically present	Treated as `NaN`; backfilled from `event_history`
`impact_level`	Source did not carry an impact label	Remains `NaN`
`signal`	Computable only when `effect_rule`, `actual_final`, and `forecast_final` are all non-null	`NaN` for all speech-like events and ~16% of quantitative events; filled with `0` in `df_daily`

Key finding on signal missingness: Non-quantifiable events (speeches, no numeric values) have 100% NaN signal by construction. Among quantitative events, ~16% are

missing signal due to absent actual or forecast data. Missing signal is therefore not random — it is structurally explained by the event type. Replacing with `0` (neutral signal) is semantically valid.

Typical non-NaN fill values in raw string columns

The `parse_value` function treats the following raw strings as `NaN` before numeric conversion: `""` (empty), `"NULL"`, `"NONE"`, `"NaN"`. These are artefacts from the scraping layer, not genuine data.

OHLC / Final Dataset

Issue	Rate	Handling
Missing `open`/`close` after merging events with OHLC	~8% of event-date/currency rows	Dropped (`dropna(subset=["return", "volatility"])`); confirmed to correspond to non-trading days (public holidays)

Erroneous Values and Data Quality Notes

- **Currency `ALL`:** Albanian Lek appeared in the events data but has no corresponding OHLC pair. Rows with `currency == "ALL"` are removed (confirmed erroneous/scraping artefact).
- **Raw value multipliers:** The `parse_value` function resolves unit suffixes (`K` = ×1,000; `M` = ×1,000,000; `B` = ×1,000,000,000) and percentage signs (`%` → divide by 100). It also normalises the Unicode minus sign (`−`) to ASCII (`-`). This is necessary because the original scraped strings are heterogeneous across event types.
- **Outliers in volatility:** IQR-based outlier bounds are computed but **not removed**. Extreme volatility spikes are the primary phenomenon of interest; removing them would eliminate the most analytically relevant observations.
- **Duplicate spec labels:** The `event_specs` table sometimes contains multiple rows for the same `event_id` / label combination (e.g., multiple `ff_notes`). The pivot uses `aggfunc="first"` to deduplicate, retaining only the first occurrence.

Covered Currencies

The analysis covers eight major currencies:

Currency	Note
AUD	Australian Dollar — higher baseline volatility
GBP	British Pound
USD	United States Dollar — proxied via inverted EUR/USD
CAD	Canadian Dollar — lowest baseline volatility
NZD	New Zealand Dollar — highest signal rule accuracy (62%)
EUR	Euro
JPY	Japanese Yen — notable isolated volatility spike mid-2024
CHF	Swiss Franc — high cross-currency spillover effect from its news

Key EDA Findings

- **Macro events** produce the highest average absolute daily return (~0.0043), followed closely by rates, inflation, and labor events.
- **AUD and NZD** exhibit the highest baseline volatility; **CAD** is the most stable.
- **Cross-currency spillovers:** High-impact CHF news events elevate volatility in other currency pairs more than any other currency. CAD news produces consistently below-baseline cross-pair effects.
- **Directional signal accuracy:** The ForexFactory "usual effect" rule is marginally predictive (53–55% accuracy) for most currencies. NZD is the exception at 62%, where the rule is meaningfully above chance.
- **Temporal structure:** Volatility follows a clear multi-month cycle across the ~2-year dataset. Models should account for regime shifts in the macro environment, as the same news event produces different market reactions depending on prevailing conditions.

Dependencies

pandas
numpy
sqlalchemy
psycopg2
matplotlib
seaborn
scipy
re (stdlib)